DATASCI 350 - Data Science Computing

Final Project Instructions

1 Overview

The final project is worth 20% of your course grade. You will work in groups of four to five students and build a reproducible container pipeline from end to end.

Your pipeline does four things. It collects data from a web API with `requests`. It processes that data with pandas or DuckDB. It presents the results in a Quarto report. It ships inside a Docker container that anyone can run.

You submit a GitHub repository. The repository holds your Quarto source, all of your code, and a Dockerfile.

Here is the rule that matters most. *If* `docker run` *on my machine does not reproduce your report, your project does not exist.* Reproducibility is the point of this course, so it is the point of this project.

DuckDB should do the main work when the analysis is SQL-shaped: joins, group-bys, and filters over a file. Pandas is fine for the rest. Dask is allowed only if your data does not fit in memory, which an API pull will not reach (see Lecture 21).

2 Timeline

The project is released on Tuesday 27 October 2026. Please email me the names of your group members by Thursday 5 November 2026, and I will keep the list myself, so there is nothing to register on Canvas. If I have not heard from you by then, I will assign you to a group at random. If you are taking Track B, please mention the API you have in mind in the same email.

The project is due on Tuesday 8 December 2026 at 11:59pm. That is the last day of class. The deadline is firm.

3 Two tracks

You choose one of two tracks. Both tracks are graded with the identical rubric. Choosing your own API earns you no bonus and costs you nothing.

3.1 Track A: the World Bank API

This is the guided track and the default choice. You use the World Bank Indicators API, which needs no key and no registration.

You pick the indicators and the countries. You build your own research question around them. For example, you might ask how under-five mortality has moved with health spending across Latin America since 2000, or whether secondary school enrolment tracks GDP per capita in Sub-Saharan Africa.

3.2 Track B: bring your own API

You may use any other public API. Please talk to me about the API you have chosen and get my approval before you start building on it.

Your API must meet three criteria:

- It is free to access.
- It needs no heavy authentication. An API key is fine. OAuth is not.
- It is documented well enough that another student in this class could use it.

Send me the API name, a link to its documentation, and two sentences on what you plan to ask of the data.

4 The report

Write the report in Quarto. Aim for 1,500 to 2,500 words, excluding code. Short and clear beats long and padded.

Your report must contain the following:

- A research question, stated in the opening paragraph.
- At least two visualisations.
- The code chunks that produce your analysis and your visualisations, visible in the rendered report rather than hidden.
- Descriptive or statistical analysis appropriate to the question you asked.
- An honest discussion of the limitations of your data and your analysis.
- A short contribution statement at the end, as described in the grading rubric section.

The limitations section is not a formality. Missing country-years, indicators that measure something narrower than their name suggests, and correlations you cannot read causally all belong there.

A word on the statistics, because this worries students every year. I do not grade the sophistication of your method, and many of you have not taken advanced statistics. Careful descriptive work, means and trends and well-chosen comparisons, correctly interpreted, scores better than a regression you cannot explain. When the rubric says analysis quality, it means three things: the method suits the question you asked, you carried it out correctly, and you read the result for what it actually shows. A group that computes a simple summary and interprets it honestly can earn full marks.

5 The container

Your repository must include a Dockerfile that builds an image containing every dependency your analysis needs. Anyone who clones your repository should be able to build the image and render the report with no extra installation.

You do not start from nothing. A minimal starter repository is available at `https://github.com/danilofreire/datasci350-project-starter`. It contains a working Dockerfile, a pinned `requirements.txt`,

a script that pulls data from the World Bank API, and a skeleton `report.qmd` whose headings match the rubric below. Clone it, check that it renders on your machine, then replace the example with your own work.

5.1 The data workflow

Your project has two separate steps, and keeping them separate is what makes the container reproducible.

The first step is the pull. A script in your repository calls the API once and saves what comes back into `data/raw/`. Save two things: the raw response exactly as the API sent it, and the tidy table you built from it. You then commit both files to the repository. The starter repository does this in `scripts/pull_data.py`.

The second step is the report. Your Quarto file reads only the saved copy in `data/raw/`. It never calls the API while rendering.

The reason is that an API is not under your control. It may change its format, rename an indicator, or simply be down on the day I mark your project in December. A report that calls the API while rendering breaks when any of that happens, and a report that does not render scores nothing for reproducibility. With the snapshot committed, the container produces the same report on any machine, in any month, with no internet connection at all.

So pull once, commit the snapshot, and leave it alone. Do not add `data/raw/` to your `.gitignore`.

5.2 Testing your container

Test your container long before you submit. Clone your own repository into a fresh folder. Build the image from that clone. Run the container and confirm the report renders. Do not test only on the machine where you wrote the code, because that machine already has your dependencies installed.

6 How to submit

1. Push your final work to your group's GitHub repository.
2. Make the repository public, or add me as a collaborator.

3. Confirm that the Dockerfile, the Quarto source, and all code are in the repository.
4. Confirm that the report renders from a clean `docker run`.
5. Post the repository link on Canvas before 11:59pm on Tuesday 8 December 2026.

One member posts the link on behalf of the group.

7 Grading rubric

Component	**Weight**
Reproducibility: the container builds and the report renders from a clean `docker run`	30%
Analysis quality: the research question, the method, and the interpretation	30%
Communication: clarity of the writing and the visualisations	20%
Code quality: organised, readable, no dead code	10%
Git workflow: meaningful commits from all members, sensible repository structure	10%
Total	**100%**

Every member must contribute to the shared repository. A member with no meaningful contribution, judged from the commit history, receives a zero for the project. Contributions must also be broadly comparable across the group: where the history shows that one member did most of the work, I will adjust individual grades to reflect each member's share. I read the history itself rather than applying a formula, so commits and lines changed count as evidence, not as a score. Your report must end with a short contribution statement saying who did what. I will check it against the repository.

8 Use of AI

The course AI policy applies to this project without change. You may use AI tools, and you must attribute them.

Add a short note at the end of your report saying how you used AI. Name the tools. Say which parts of the work they touched.

Every group member must be able to explain any part of the submission. If you cannot explain a block of code in your own repository, do not submit it.

9 Questions

Please bring your questions to class or to office hours. Ask about the container early: Docker problems are much easier to solve with a few days to spare than on the night of the deadline.